Reviewer Pack — Minimal Rank of Commutative Quadratic Forms Bounds BP Read-T…

Entry b670ff06f940 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 06:11:54 UTC

Minimal Rank of Commutative Quadratic Forms Bounds BP Read-Twice Size

Entry ID: `b670ff06f940` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 06:11:54 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Commutative Quadratic Forms) **Field B** (complexity object): Complexity Theory: Branching Program Complexity

Statement:

> ['For every read-twice BP P with m variables and s clauses, the minimal rank of its associated quadratic form matrix Q(P) is bounded by O(m^2 log n), where n = 2m + s.', 'For a trivial inner product BP IP_2 on n variables, the minimal rank of its quadratic form matrix Q(IP_2) is Ω(n).', 'The conjecture is falsified if there exists a read-twice BP P with m variables and s clauses such that the rank of Q(P) is O(m^2 log n) but the size of P is O(n^{1.5}).']

Rationale (proposer's reasoning):

> ['Commutative quadratic forms provide a rich algebraic structure to study symmetries and constraints in systems, which might reflect the complexity of read-twice BP computations.', 'Previous work has shown that certain properties of quadratic forms are related to circuit complexity, suggesting their potential use as complexity-theoretic invariants.', 'The conjecture aims to bridge the gap between algebraic combinatorics and branching program complexity, potentially revealing new insights into the computational power of read-twice BP.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f5f5b32cea610346`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across a statistically significant number of seeds (e.g., n=100), the mean minimal rank of the quadratic form matrices does not exceed O(m^2 log n) AND there is no instance where the size of P exceeds O(n^{1.5}) given that its minimal rank is within O(m^2 log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Commutative Quadratic Forms AND Branching Program Complexity - Quadratic Form Matrix Rank Bound read-twice BP AND m variables s clauses - Inner Product BP IP_2 Minimal Rank Ω(n) AND read-twice BP size O(n^{1.5})

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            raise ValueError("Matrix is singular")
        for j in range(i + 1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] += factor * A[i][k]
    return A

def rank(A):
    A = [row[:] for row in A]
    r = gaussian_elimination(A)
    return sum(1 for row in r if any(row))

def quadratic_form_matrix(bp):
    n = bp['n']
    m = len(bp['clauses'])
    Q = [[0] * (2 * n) for _ in range(n)]
    for clause in bp['clauses']:
        x = [bp['variables'].index(var) for var in clause]
        for i in x:
            Q[i][i + n] = 1
```

```
            for j in x:
                if i != j:
                    Q[i][j + n] += 1
    return Q

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        m = random.randint(1, n)
        s = random.randint(1, n)
        variables = ['x' + str(i) for i in range(n)]
        clauses = []
        for _ in range(s):
            clause = random.sample(variables, random.randint(1, n))
            clauses.append(clause)
        bp = {'n': n, 'variables': variables, 'clauses': clauses}
        Q = quadratic_form_matrix(bp)
        rank_Q = rank(Q)
        total_rank += rank_Q
        instances_tested += 1

        if rank_Q > m**2 * math.log(n):
            conjecture_holds = False
            counterexample = f"Rank {rank_Q} exceeds O(m^2 log n) for n={n}, m={m}"

    mean_rank = total_rank / instances_tested
    return {
        "metric_name": "Minimal Rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r['metric_value'] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)
    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
```

```
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f137099b.py", line 89, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f137099b.py", line 67, in run_trial
    rank_Q = rank(Q)
             ^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f137099b.py", line 33, in rank
    r = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f137099b.py", line 24, in gaussian_elim
    raise ValueError("Matrix is singular")
ValueError: Matrix is singular
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that the test can be completed and the conjecture can be properly evaluated.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12182
2	preregistration	ollama_remote	glm4:latest	0	0	6126
3	novelty	ollama_remote	glm4:latest	0	0	4881
4	novelty	ollama_remote	glm4:latest	0	0	6519
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16140
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10407
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10460
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10417

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	10545

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 87676 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b670ff06f940.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b670ff06f940.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b670ff06f940.tar.gz` (if generated)